

FVRULELEARNER: Operator-Level Reasoning Tree (OP-TREE)-Based Rules Learning for Formal Verification

Lily Jiaxin Wan^{1,*}, Chia-Tung Ho^{2,*}, Yunsheng Bai², Cunxi Yu^{2,3}, Deming Chen¹, Haoxing Ren²

¹University of Illinois Urbana-Champaign, ²NVIDIA, ³University of Maryland, College Park

Abstract—The remarkable reasoning and code generation capabilities of large language models (LLMs) have recently motivated increasing interest in automating formal verification (FV), a process that ensures hardware correctness through mathematically precise assertions but remains highly labor-intensive, particularly through the translation of natural language into SystemVerilog Assertions (NL-to-SVA). However, LLMs still struggle with SVA generation due to limited training data and the intrinsic complexity of FV operators. Consequently, a more efficient and robust methodology for ensuring correct SVA operator selection is essential for producing functionally correct assertions. To address these challenges, we introduce FVRULELEARNER, an Operator-Level Rule (Op-Rule) learning framework built on a novel Operator Reasoning Tree (OP-TREE), which models SVA generation as structured, interpretable reasoning. FVRULELEARNER operates in two complementary phases: (1) Training: it constructs OP-TREE that decomposes NL-to-SVA alignment into fine-grained, operator-aware questions, combining reasoning paths that lead to correct assertions; and (2) Testing: it performs operator-aligned retrieval to fetch relevant reasoning traces from the learned OP-TREE and generate new rules for unseen specifications. In the comprehensive studies, the proposed FVRULELEARNER outperforms the state-of-the-art baseline by 3.95% in syntax correctness and by 31.17% in functional correctness on average. Moreover, FVRULELEARNER successfully reduces an average of 70.33% of SVA functional failures across diverse operator categories through a functional taxonomy analysis, showing the effectiveness of applying learned OP-TREE to the Op-Rule generations for unseen NL-to-SVA tasks. These results establish FVRULELEARNER as a new paradigm for domain-specific reasoning and rule learning in formal verification. The source code and benchmark are available at <https://github.com/NVlabs/FVRuleLearner>.

Index Terms—Large Language Models (LLMs), Formal Verification, SystemVerilog Assertions (SVAs)

I. INTRODUCTION

In the rapidly evolving hardware industry, the increasing complexity of integrated circuits (ICs) poses significant challenges to the verification process, a critical phase that accounts for nearly half of the total design cycle [1]. Traditional simulation-based verification struggles to achieve comprehensive coverage, motivating the rise of formal verification (FV) as a complementary methodology that expresses design intent as temporal logic properties and exhaustively proves their correctness under all possible input conditions [2], [3]. However, writing SVAs from natural-language specifications (NL-to-SVA) remains a major bottleneck: it is labor-intensive, error-prone, and requires substantial expertise in both hardware design and formal methods. Despite extensive literature and standardized assertion libraries [4], [5], [6], [7], the manual construction of assertions is still tedious and prone to mistakes, which has naturally motivated the development of automated assertion-generation methods, including recent LLM-based approaches. Unfortunately, current LLM systems still fall short in both accuracy and reliability for SVA generation. Fig. 1 illustrates an example of these limitations.

The challenges in translating NL specifications to SVAs (NL-to-SVA) have sparked interest in applying Large Language Models (LLMs) to automate and improve productivity in digital design

*Corresponding authors: Lily Jiaxin Wan (wan25@illinois.edu) and Chia-Tung Ho (chiatungh@nvidia.com).

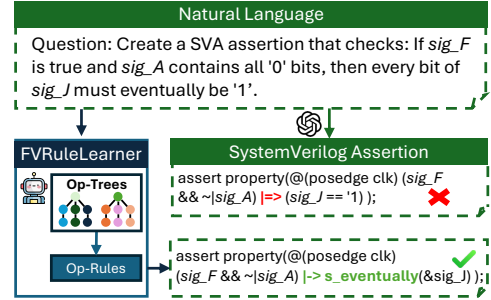


Fig. 1: Given a NL-to-SVA task, LLM inference often produces functionally incorrect SVAs, whereas our FVRULELEARNER framework leverages learned OP-TREE to generate functionally correct SVAs.

verification [8], [9], [10]. Despite their proficiency in generating code across various languages [11], [12], their performance on formal verification tasks remains limited. Generated assertions often suffer from syntactic violations [13], [14] and, even when syntactically correct, frequently misrepresent the intended functional semantics [15], often failing to produce accurate SVA operators, especially for temporal implication and delay. Consequently, a more efficient and robust methodology to ensure correct SVA operator selection is essential for generating functionally correct SVAs.

We propose FVRULELEARNER, an operator-level rule learning framework that models assertion generation as a structured reasoning process over formal operators using novel **Operator Reasoning Tree** (Op-Tree). FVRULELEARNER constructs operator-level reasoning traces using the proposed OP-TREE during training. At testing time, these reasoning traces are retrieved and transformed into Operator-level Rules (Op-Rules) through a rule-adaptation process that enables effective generalization for guiding SVA generation. Our contributions are summarized below:

- 1) We developed an operator-level rule learning framework for SVA generation with the novel OP-TREE, that models assertion generation as structured reasoning. These learned operator-level reasoning traces are retrieved to generate operator-level rules in the testing time, which enables LLMs to adapt and continuously improve on unseen examples without relying on expensive model fine-tuning.
- 2) We perform comprehensive studies to demonstrate the superior performance of FVRULELEARNER on functional correctness across the three datasets. FVRULELEARNER outperforms the state-of-the-art baseline by 3.95% in syntax correctness and by 31.17% in functional correctness on average.
- 3) We conduct a functional taxonomy analysis and demonstrate that FVRULELEARNER significantly reduces functional mismatches, achieving an average 70.33% reduction in SVA functional failures across diverse operator categories relative to the state-of-the-art baseline.
- 4) We develop and release a new dataset of 1,000 NL-to-SVA pairs from open-source hardware designs [16], establishing a reproducible benchmark for LLM-assisted formal verification.

II. RELATED WORK

A. Formal Verification

Early attempts at automated FV assertion generation using template-based or traditional ML methods faced limitations in handling the ambiguity and compositional structure of natural language specifications [17], [18], [19]. Benchmarking efforts such as FVEval [20] and AssertionBench [21] have established the first systematic evaluation protocols for LLMs in formal verification. Building on these benchmarks, recent studies have explored LLMs for NL-to-SVA translation, showing progress through document- or RTL-aware prompting and multi-stage reasoning pipelines, yet challenges remain in model generalization and explainability [9], [22], [10], [15], [23], [24], [25]. The work by Orenes-Vera et al. [10] manually designed rules to guide GPT-4 in generating better SVAs, whereas our approach focuses on learning from training data to automate this process. More recently, Xiao et al. [26] combine retrieval-augmented generation (RAG) with task-specific fine-tuning to improve the alignment between natural language specifications and generated assertions.

B. LLM for Code and Hardware Design

LLMs have shown notable proficiency in code generation across various programming languages [27], [28], [29], [30], [31], [32], [33], with advancements like Claude-Sonnet-4.5 [34] and DeepSeek-Coder-V2 [35] leading the way. Recent efforts in Electronic Design Automation (EDA) have leveraged LLMs for tasks such as HDL generation and analysis [8], [36], [37], [38], yet the scarcity of high-quality, domain-specific datasets remains a barrier [39]. However, their methodologies are limited and cannot be directly applied to the formal verification task [9], [22], [10].

C. LLM Self-Learning and Learning from Mistakes

Studies have introduced LLM-based systems incorporating self-learning, self-correction, learning from previous mistakes through various approaches [40], [41], [42]: supervised fine-tuning with reinforcement learning for code refinement [43], iterative refinement with self-feedback [44], simulators for self-correction in hardware design [45], linguistic feedback for trial-and-error learning [46], Learning-From-Mistakes Prompting for low-resource language translation [47], recursive introspection [48], retrieved in-context principles from previous mistakes [49], [50], [51], pseudo-graph RAG for knowledge indexing and retrieval [52], dual-agent system [53], agent and prompt optimization [54], [55], [56], [57], etc. Unlike methods on model tuning or test case clustering, our approach first validates the effectiveness of learning operator-level reasoning traces during training and subsequently applies these Op-Rules to unseen cases.

III. PRELIMINARIES

A. Problem Definition

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ represent the dataset, where x_i is a NL specification and y_i is the corresponding golden SVA. Given an NL-to-SVA task, we define the LLM inference as a function $F : \mathcal{X} \rightarrow \mathcal{Y}$, which maps NL specifications $x \in \mathcal{X}$ to predicted SVAs $\hat{y} = F(x) \in \mathcal{Y}$. The goal is to generate $\hat{y}$ that matches the golden SVA y , evaluated by metric M measuring **syntax correctness**, **functionality**, and **linguistic similarity**. Here, FVRULELEARNER learns the operator-level reasoning traces to generate the Op-Rules $\mathcal{R}$ for maximizing metric M for NL-to-SVA tasks, formulated as

$$\mathcal{R}^* = \arg \max_{\mathcal{R}} \mathbb{E}_{(x,y) \sim \mathcal{D}} [M(y, \hat{y}) \mid \mathcal{R}]. \quad (1)$$

TABLE I: Operator Categories and SystemVerilog Operators Table.

Operator Category	Representative Operators
Temporal Implication Operators	->, ==>
Temporal Delay Operators	##m, [m:n]
Temporal Sampling Operators	\$past, \$rose, \$fell, \$stable
Temporal Liveness Operators	s_eventually, s_always, strong
Combinational Logic Operators	&&, , !, ==, !=, ^
Miscellaneous Differences	@(posedge clk), disable iff (rst)

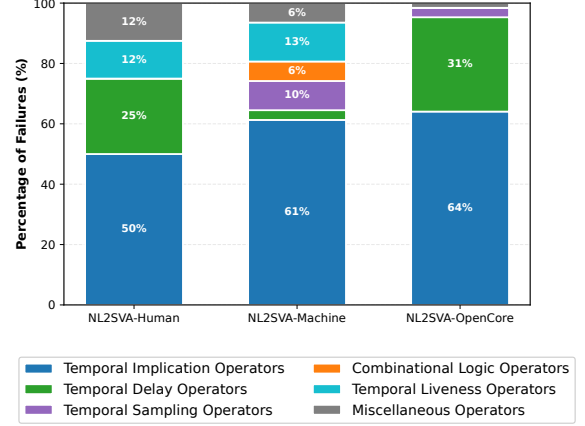


Fig. 2: Operator-level functional mismatch analysis between generated and golden SVAs across three datasets. Temporal operators dominate the errors, highlighting the need for operator-aware reasoning.

B. Operator-Level Functional Mismatch Analysis

We categorize SVA operators into six groups: temporal implication, temporal delay, temporal sampling, temporal liveness, combinational logic, and miscellaneous structural forms, as shown in Table I. We further analyze the functionally incorrect cases identified by the equivalence-checking tool to determine the operator-level failure modes. We classify operator mismatches based on the generated SVA's operator rather than the golden SVA, explicitly highlighting the specific operators the model tends to hallucinate.

Fig. 2 shows that **temporal operators dominate functional mismatches in the NL-to-SVA task**, contributing to more than 80% of all failures across datasets. Among them, temporal implication operators ($|->$, $|==>$) alone cause 50–64% of errors, showing that LLMs often confuse same-cycle and next-cycle semantics. Temporal delay and sampling operators introduce additional errors, indicating that LLMs struggle to represent timing intervals and signal transitions. In contrast, combinational mismatches are rare, suggesting that LLMs handle Boolean logic more effectively. Overall, these findings indicate that the core difficulty in NL-to-SVA translation stems from insufficient temporal reasoning, highlighting the importance of operator-aware learning that captures event ordering and timing semantics.

C. Operator-Level Rule (Op-Rule)

Unlike general rules which offer broad advice, we tackle the challenges of using LLMs to understand operators, an Operator-Level Rule (Op-Rule) is a granular, executable directive targeting specific SVA constructs. We define an Op-Rule as a structured directive that explicitly maps a specific operator context to a precise syntactic modification. As illustrated in Fig. 3, the General Rule fails because it relies on abstract guidance to "align assertion syntax with intended temporal logic." Since LLMs struggle with the precise semantics of

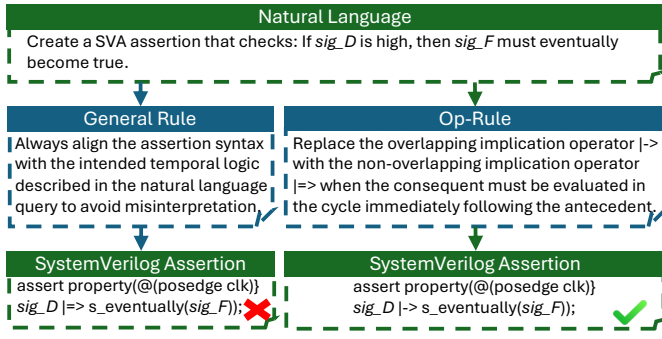


Fig. 3: Comparison between General Rules and Op-Rules.

temporal operators, this ambiguity leads to incorrect SVA generation. In contrast, the Op-Rule succeeds by enforcing execution over interpretation. By implementing a deterministic, symbol-level directive rather than relying on semantic reasoning, the Op-Rule ensures the generation of the correct overlapping implication.

IV. PROPOSED APPROACH: FVRULELEARNER

We propose FVRULELEARNER, a novel framework that shifts the generation paradigm from monolithic text translation to explicit operator-level reasoning, effectively bridging the gap between natural language and the strict timeline semantics of SystemVerilog. As illustrated in Fig. 4, our approach decomposes this challenge into two phases: a **Training Phase** that acts as a self-reflective engine to distill high-fidelity reasoning traces (Op-Trees) from failure cases, and a **Testing Phase** that uses a hybrid retrieval mechanism to enforce structural alignment on unseen specifications. As a result, the dataset is divided into: (1) **Training set**: $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{N_{\text{train}}}$, and (2) **Testing set**: $\mathcal{D}_{\text{test}} = \{(x_i, y_i)\}_{i=1}^{N_{\text{test}}}$, where N_{train} and N_{test} are the sizes of the training and testing sets, respectively.

A. Training Phase

Given the training set $\mathcal{D}_{\text{train}}$, the framework utilizes the golden SVA as an oracle to diagnose mismatches and invokes *Op-Tree reasoning* to derive corrective operator-level steps. The validated traces form the verified Op-Tree set $\mathcal{T}$, which conditions the model through retrieved and regenerated operator rules for unseen inputs. The training objective is therefore to learn the optimal Op-Tree reasoning traces:

$$\hat{y} = F_{\mathcal{T}}(x) = F(x, r(x, y, \mathcal{T}))$$

$$\mathcal{T}^* = \arg \max_{\mathcal{T}} \mathbb{E}_{(x, y) \sim \mathcal{D}_{\text{train}}} [M(y, F_{\mathcal{T}}(x))] \quad (2)$$

Here, r is the function to generate the Op-Rules $\mathcal{R}$ from the reasoning traces of OP-TREE ($\mathcal{T}$) with the NL (x), golden SVA (y). $F(x, r)$ represents the LLM inference with NL and the corresponding $\mathcal{R}$ as input and outputs the generated SVAs.

Operator Reasoning Tree (OP-TREE): To diagnose assertion failures, we introduce the Operator Reasoning Tree (OP-TREE), a hierarchical reasoning structure generated via operator-directed prompting. Unlike linear chain-of-thought, the OP-TREE mimics the systematic debugging workflow of a verification engineer, decomposing the error into three orthogonal layers (as shown in Fig. 4(c):

- 1) **Contextual Diagnosis:** Analyzes specific signal relationships in the design to formulate a hypothesis about the failure mode.
- 2) **Theoretical Grounding:** Validates this hypothesis by retrieving standard definitions (e.g., “Formal distinction between Overlapping ‘ $\rightarrow$ ’ and Non-Overlapping ‘ $\Rightarrow$ ’ implication”), anchoring the reasoning to formal axioms.

- 3) **Rule Generation:** Synthesizes the specific context and theoretical grounding into a correction Op-Rule at the leaf node.

To ensure broad reasoning coverage, we let LLMs focus on different aspects, to detect and discourage repetitive questions. This encourages the model to explore different error categories such as timing or logic rather than simply rephrasing the same hypothesis.

Training Iterations: To guarantee the reliability of the learned rules, we enforce a strict acceptance criterion. As shown in Iteration 2 of Fig. 4, the rules extracted from the OP-TREE are fed back into the system to regenerate the assertion (SVA_{1,2}). One OP-TREE is deemed “Valid” and committed to the database if and only if this rule-guided iteration produces an SVA that is functionally equivalent to the golden reference (marked by the green checkmark ✓). This self-reflection process effectively ensures the system learns exclusively from empirically verified correction strategies.

B. Testing Phase

The testing phase executes a dynamic, retrieval-augmented inference pipeline to adapt learned reasoning to unseen specifications. For a given input $x \in \mathcal{D}_{\text{test}}$, we firstly generate an initial SVA $\hat{y}_{\text{init}}$ without Op-Rules. We then retrieve the relevant learned OP-TREE $\bar{T}$ from $\mathcal{T}^*$ by selecting the top-k similarity score of the x , and the background node of each learned OP-TREE. The reasoning traces τ are extracted from root node to leaf nodes of each OP-TREE in $\bar{T}$. Then, the top-k reasoning traces τ^* are selected using a novel hybrid score of operator and semantic by referencing $\hat{y}_{\text{init}}$ and x for generating Op-Rules $\mathcal{R}^*$ in rule adaption phase ($ra(x, \tau^*)$) for generating the final SVA for x in Eq. 3.

$$\hat{y} = F(x, \mathcal{R}^*) = F(x, ra(x, \tau^*)), \tau^* \in \bar{T} \quad (3)$$

Here, ra denotes the rule adaptation procedure that transforms the selected reasoning traces into the Op-Rules $\mathcal{R}^*$. We empirically use $k = 3$. We will introduce the novel hybrid score that considering operator and semantic for reasoning traces selection and rule adaption below.

Hybrid Score for Reasoning Traces Selection: Standard RAG systems rely on linguistic similarity, which is often a poor proxy for verification logic (e.g., keywords match but semantics differ). To bridge this gap, we evaluate every candidate reasoning trace τ considering operator and semantic as Eq. 4.

$$S_{\text{hybrid}} = \begin{cases} 0, & \text{if } S_{op} = 0 \text{ or } S_{llm} = 0 \\ \alpha \cdot S_{op} + (1 - \alpha) \cdot S_{llm}, & \text{otherwise} \end{cases} \quad (4)$$

where α is a hyperparameter balancing operator and semantic weights. We empirically set $\alpha = 0.5$ for the experiments. The components are defined as follows:

- **Operator Alignment Score (S_{op}):** This metric quantifies operator relevance between the candidate reasoning traces and $\hat{y}_{\text{init}}$. Let $\mathbb{O}$ denote the universe of valid operators defined in Table I. We define the operator extraction function Φ as the projection of each reasoning trace τ^* and $\hat{y}_{\text{init}}$ in text format onto a operator subset $\mathcal{O} \in \mathbb{O}$. The extracted operator subset of $\hat{y}_{\text{init}}$, and τ^* are $\mathcal{O}_{\text{init}} = \Phi(\hat{y}_{\text{init}})$, and $\mathcal{O}_{\text{trace}} = \Phi(\tau^*)$, respectively. To robustly handle operator variations (e.g., matching $\rightarrow$ with $\Rightarrow$), we compute the alignment using a *soft Jaccard Similarity*:

$$S_{op}(\tau^*) = \frac{\sum_{u \in \mathcal{O}_{\text{init}}} \max_{v \in \mathcal{O}_{\text{trace}}} \text{sim}(u, v)}{|\mathcal{O}_{\text{init}} \cup \mathcal{O}_{\text{trace}}|} \quad (5)$$

Here, $\text{sim}(u, v) \in [0, 1]$ assigns 1 to identical operators, 0.5 to same category but not identical one (Table I), and 0 otherwise.

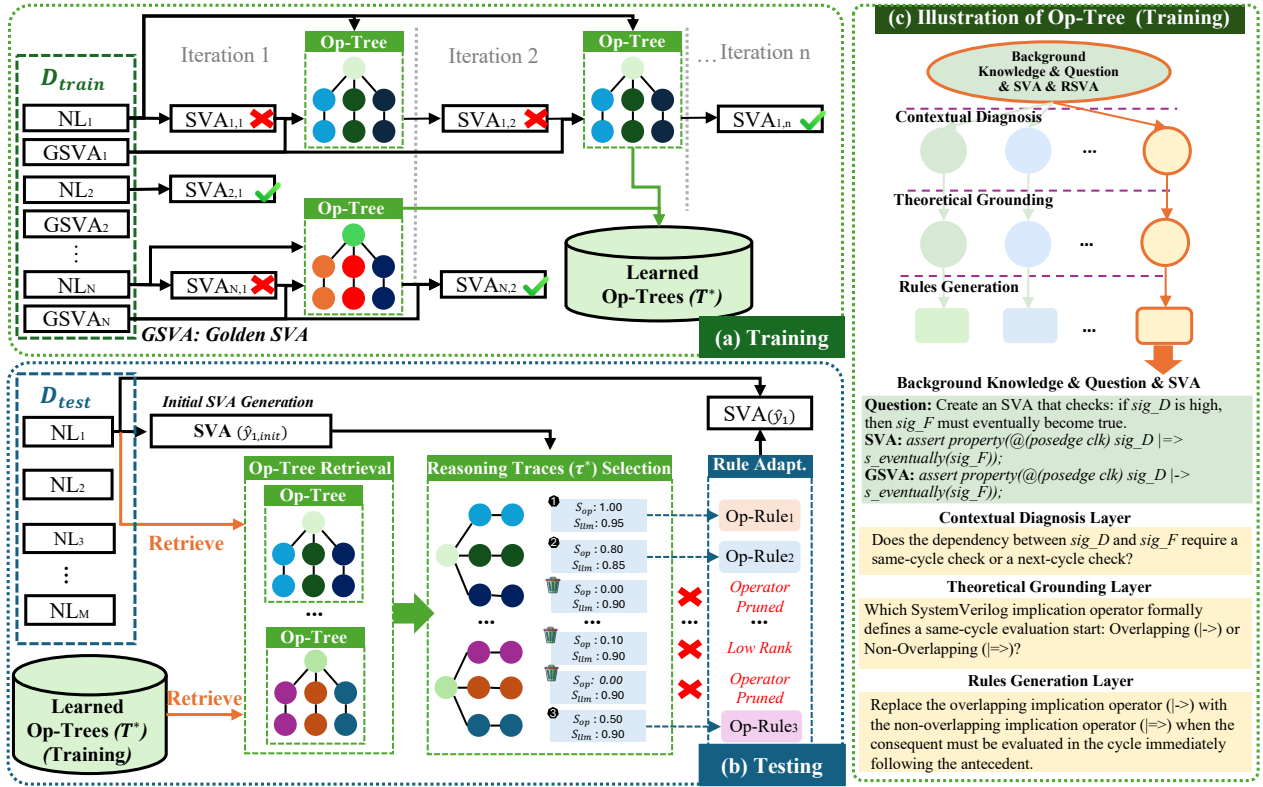


Fig. 4: The flow overview of FVRULELEARNER and an illustration of OP-TREE in training. (a) Training phase: The framework leverages OP-TREE to consolidate the operator-level reasoning traces and they are stored in T^* . (b) Testing phase: FVRULELEARNER firstly generates initial SVA ($\hat{y}_{init}$) and retrieve relevant OP-TREE from T^* according to the similarity score of natural language specification of testing data and the background nodes' natural language content. Then, the candidate reasoning traces are selected based on the proposed operator alignment and semantic applicability scores. Lastly, the rule adaption step generate Op-Rules for final SVA ($\hat{y}$). (c) An **illustrative example** of the Op-Tree reasoning process, showing how hierarchical decomposition links design context to theoretical definitions to produce a verified correction rule.

- **Semantic Applicability Score (S_{sim}):** This metric ensures reasoning quality. We employ an LLM-as-a-judge to evaluate the reasoning chain's transferability [58]. We employ an LLM-as-a-judge [58] to assign a confidence score $S_{sim} \in [0, 1]$ for each reasoning trace τ^* and natural language specification x .

The final ranking ensures that selected reasoning traces τ^* are both linguistically relevant and operator aligned.

Rule Adaption: While retrieved reasoning traces τ^* capture high-fidelity reasoning patterns, they remain semantically coupled to the specific signal names or parameters (i.e., clock cycle) from training dataset D_{train} . To enable generalization at testing time, we apply a signal abstraction mechanism that masks instance-specific variables with generic placeholders (e.g., $\langle signal \rangle$), enabling the model to focus on operator-level reasoning. This step injects formal definitions directly into the context window, enforcing strict adherence to SystemVerilog semantics. Guided by this context, the LLM generates new Op-Rules $\mathcal{R}^*$ adapted to the target specification to generate the final SVA as mentioned in Eq. 3.

V. EXPERIMENTS

In this section, we present the comprehensive experimental results of applying our framework to various Large Language Models (LLMs) and compare their performance against state-of-the-art results from existing literature.

A. Experimental Settings

Dataset: Our experiments were conducted using three distinct datasets: NL2SVA-HUMAN and NL2SVA-MACHINE [20] are

specifically curated for natural-language to SVA generation tasks. In addition, we include NL2SVA-OPENCORE, a new dataset derived from ASSERTIONBENCH [21], consisting of 1,000 NL-SVA pairs mined from real-world OpenCore designs, targeting SVA generation for module interfaces. Each dataset is randomly split into 80% training and 20% testing subsets to ensure rigorous comparison and evaluation of the models' performance.

Evaluation Metrics: We use four key metrics: (1) BLEU: Measures the textual similarity between generated and reference assertions, ranging from 0 (no similarity) to 1 (perfect match); (2) SYN: Evaluates the syntactic correctness of generated assertions (0 or 1); (3) FUNCTIONALITY: Strictly evaluates whether the generated assertion's behavior precisely matches the reference (0 or 1); (4) RELAXED FUNCTIONALITY: Evaluates functional equivalence, allowing for implication relationships between the generated and reference assertions (0 or 1). An implication relationship here means if the reference assertion holds, the generated assertion must also hold, allowing for more permissive generated assertions. JASPER [59] is further used as both the syntax checker during the testing phase and as an evaluation tool for assessing assertion functional correctness with two sets of scripts.

Baselines: We compare FVRULELEARNER against several state-of-the-art baselines with publicly available implementations, including the zero-shot (FVEVAL-0-SHOT) and few-shot (FVEVAL-3-SHOT) prompting configurations from FVEval [20]. Additionally, we evaluate an ENSEMBLE [60], which aggregates multiple inference passes from the base model using majority voting to improve robustness.

TABLE II: Experimental Results of LLM Models on NL2SVA-HUMAN, NL2SVA-MACHINE, and NL2SVA-OPENCORE Testing Sets.

Model / Method	NL2SVA-HUMAN				NL2SVA-MACHINE				NL2SVA-OPENCORE			
	BLEU	SYN	FUNC	R.FUNC	BLEU	SYN	FUNC	R.FUNC	BLEU	SYN	FUNC	R.FUNC
GPT-4o												
FVEVAL-0-SHOT	0.5203	0.9333	0.5333	0.6667	0.6054	0.9500	0.4167	0.4500	0.6145	0.9250	0.7050	0.7200
FVEVAL-3-SHOT	0.5823	0.9333	0.4000	0.6000	0.6461	0.9500	0.4333	0.5167	0.6187	0.8950	0.6750	0.6950
ENSEMBLE	0.5876	0.8667	0.4000	0.4667	0.6562	0.9667	0.5000	0.6000	0.5922	0.9550	0.7000	0.7150
FVRULELEARNER	0.6846	1.0000	0.8000	0.8000	0.7072	0.9667	0.7500	0.8167	0.6976	0.9900	0.8500	0.8750
Claude 4.5 Sonnet												
FVEVAL-0-SHOT	0.5330	1.0000	0.6000	0.8667	0.6246	0.9167	0.5333	0.6167	0.5868	0.9350	0.6900	0.7000
FVEVAL-3-SHOT	0.6480	1.0000	0.5000	0.5000	0.6765	0.9500	0.4667	0.5667	0.6154	0.9150	0.7050	0.7250
ENSEMBLE	0.5324	1.0000	0.5333	0.8000	0.6212	0.9500	0.5333	0.6333	0.5881	0.9600	0.7050	0.7250
FVRULELEARNER	0.7416	1.0000	0.8667	0.8667	0.7038	0.9667	0.7500	0.8167	0.6901	0.9700	0.8650	0.8900
o3-mini												
FVEVAL-0-SHOT	0.5886	0.9333	0.4667	0.6000	0.6075	0.9333	0.4500	0.5333	0.5900	0.9600	0.6800	0.7000
FVEVAL-3-SHOT	0.6360	0.9333	0.4667	0.5000	0.6696	0.9500	0.4833	0.5833	0.6189	0.8950	0.6800	0.7150
ENSEMBLE	0.6127	0.9333	0.4667	0.5500	0.6127	0.9333	0.4667	0.5500	0.5900	0.9600	0.6800	0.7000
FVRULELEARNER	0.7363	1.0000	0.8667	0.9333	0.6980	0.9667	0.7833	0.8333	0.7145	0.9950	0.9150	0.9300

Note: BLEU= BLEU score, SYN= Syntax, FUNC= Functionality, R.FUNC= Relaxed Functionality.

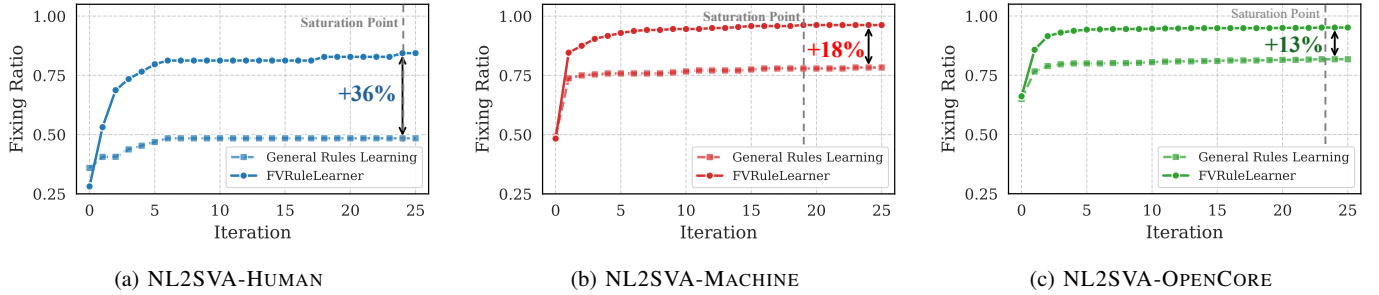


Fig. 5: Comparison of fixing ratio trends over training iterations for FVRULELEARNER (circle) versus General Rule Learning (square) methods. Across (a) NL2SVA-HUMAN, (b) NL2SVA-MACHINE, and (c) NL2SVA-OPENCORE, the proposed OP-TREE rules learning methodology consistently accelerates convergence and achieves higher final accuracy, improving the fixing rate by an average of 23.33%.

Since each baseline, as well as our FVRULELEARNER framework, requires an LLM engine for inference, we consider a diverse selection of both general instruction-tuned models and reasoning models from two major commercial LLM families: (1) GPT-4o [61], (2) CLAUDE-4.5 SONNET [34], (3) O3-MINI [62].

B. Training Phase

We show the functionality fixing ratio of the training set for each dataset over 25 iterations of the FVRULELEARNER against a general rule learning baseline in Fig. 5. The general rule learning baseline iteratively explore general rules for each NL-to-SVA problem, without OP-TREE reasoning. FVRULELEARNER demonstrates superior convergence across all benchmarks, achieving an average 23.33% improvement in fixing rates. Moreover, FVRULELEARNER achieves up to 36% functional fixing rate on NL2SVA-HUMAN. The steady performance gains across diverse specifications highlight that fine-grained operator-level reasoning effectively resolves the challenging SVA generation tasks. The remaining gap to 100% corresponds to failures arising from highly coupled global dependencies or ambiguous intent, particularly within the NL descriptions in NL2SVA-HUMAN.

Training Ratio Study: Here, we analyze the NL2SVA-MACHINE to examine the functionality score trends under varying training data ratios, leveraging its larger scale to obtain statistically meaningful

insights on testing performance. Here, we implemented an operator-level sampling strategy instead of using standard random sampling, since standard random sampling fails to capture this efficiency due to the *long-tail distribution* of industrial specifications, where critical operators (e.g., temporal liveness, sampling operators) are statistically sparse compared to trivial combinational logic. We firstly categorize the data points in the training dataset based on Table I. Then, we evenly sample the data points from each categories, and remove the duplicated data points since a data point can be in different groups since its golden SVA contains operators in different categories.

The functional correctness improves sharply, saturating at a mere 48% training ratio as shown in Fig. 6. From the study, we can observed that FVRULELEARNER performance depends on the diversity of Op-rules on operators from training stage rather than dataset scale. In summary, these results show that FVRULELEARNER is highly data-efficient, effectively generalizing from a compact, high-density set of reasoning traces without the need for large-scale data overhead for training. The maximum training data ratio is 80%.

C. Testing Phase

We show the main experimental results in Table II that demonstrates the efficacy of the FVRULELEARNER framework across three diverse datasets and various state-of-the-art LLMs. On average, our FVRULELEARNER framework achieves state-of-the-art 85.50%

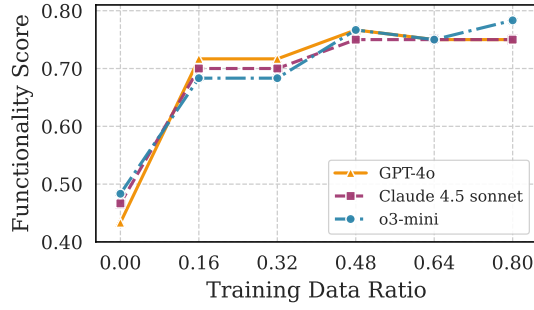


Fig. 6: Effect of Training Data Ratio on FVRULEARNER Performance on the NL2SVA-MACHINE dataset.

TABLE III: Rule Retrieval Ablation Study Results for O3-MINI on NL2SVA-MACHINE Dataset.

Model / Method	SYN	FUNC	R.FUNC
O3-MINI			
FVEVAL-3-SHOT	0.9500	0.4833	0.5833
General Rules	0.9500	0.6667	0.7500
Static Op-Rules	0.9500	0.7167	0.7833
FVRULEARNER	0.9667	0.7833	0.8333

functional correctness across three datasets using O3-MINI, representing up to a 31.17% improvement over the strongest existing baselines. For GPT-4O and CLAUDE-4.5 SONNET, FVRULEARNER further improves the average functionality score by 22.06% and 18.11%, respectively. In addition, FVRULEARNER achieves 98.39% syntax correctness and 86.24% relaxed functionality on average across the three datasets and all models. The results consistently demonstrate improvements across all evaluated metrics, underscoring the framework’s robustness and adaptability. Collectively, these results demonstrate that FVRULEARNER consistently produces high-quality assertions that are both syntactically correct and functionally accurate across various LLM models.

Rule Retrieval Ablation Study: We dissect the inference pipeline to distinguish the impact of rule *granularity* versus *adaptability*. We benchmark FVRULEARNER against two variants: (1) *General Rules*, which relies on broad rules generated during training; and (2) *Static Op-Rules*, which applies Op-Rules from training directly without rule adaption for the testing data. Adaptation is crucial because static rules often reference generic signals that are absent in the specific design context; for example, a static rule might include `tb_gnt`, which does not exist in the target design, leading to undefined signal errors if applied directly. Table III shows that operator-level specificity offers a baseline improvement over general rules, and rule adaption process further boosts the performance, pushing functional accuracy to 78.33%.

D. Runtime and Token Cost Analysis

Here, we report the runtime and token cost of O3-MINI for training, and testing. Per NL-to-SVA pair, for the NL2SVA-HUMAN dataset, training the OP-TREE takes approximately 65.36 seconds and consumes 47.6k tokens with the approximate cost of \$0.12 per case; testing one assertion takes 3.92 seconds and 11.2k tokens with the cost of \$0.03. The NL2SVA-MACHINE dataset requires 82.68 seconds (9.0k tokens, \$0.02) for training and 48.86 seconds (9.0k tokens, \$0.02) for generation. Finally, the NL2SVA-OPENCORE dataset requires 20.13 seconds (4.6k tokens, \$0.01) for training and 50.80 seconds (7.9k tokens, \$0.02) for generation. Both training and generation processes remain fully parallelizable. This shows the training procedure of

TABLE IV: Operator-Level Failure Analysis showing the reduction in functional mismatches. The “Red.” column indicates the percentage reduction, where bold blue values highlight significant improvements achieved by FVRULEARNER. “Red.” is the percentage reduction (Temp.=Temporal, Comb.=Combinational, Impl.=Implication, Sampl.=Sampling, Live.=Liveness).

Op. Category	NL2SVA-Human			NL2SVA-Machine			NL2SVA-OpenCore		
	Base	Ours	Red.	Base	Ours	Red.	Base	Ours	Red.
Temp. Impl.	4	1	-75%	19	4	-79%	41	5	-88%
Temp. Delay	2	0	-100%	1	0	-100%	20	9	-55%
Temp. Sampl.	0	0	–	3	2	-33%	2	1	-50%
Temp. Live.	1	0	-100%	4	3	-25%	0	0	–
Comb. Logic	0	0	–	2	2	0%	0	0	–
Miscellaneous	1	1	0%	2	1	-50%	1	1	0%
Total Failures	8	2	-75%	31	12	-61%	64	16	-75%

FVRULEARNER is cost efficient and can improve the testing performance significantly.

E. Functional Taxonomy Analysis at the Operator Level

We analyze the efficiency of FVRULEARNER on correcting functional failures of different operator categories (i.e., Section III-B) compared to FVEVAL-3-SHOT. In Table IV, FVRULEARNER achieves a massive reduction in functional mismatches, decreasing total failures by an average of **70.3%** across all datasets. Crucially, the improvement is most pronounced in the categories where LLMs typically struggle the most:

- **Temporal Implication Operators (80.6% Reduction):** The sharp decline in implication errors (e.g., swapping `|=>` with `|=>`) indicates that the Op-Rules successfully guide the model to distinguish between *overlapping* and *non-overlapping* validity, which is a common semantic blind spot for standard LLMs.
- **Temporal Delay Operators (85.0% Reduction):** The near-elimination of delay errors demonstrates that the retrieved rules provide precise guidance on cycle-accurate sequencing, correcting off-by-one errors that plague zero-shot generation.

This targeted reduction proves that FVRULEARNER does not simply memorize patterns, but actively aligns the LLM’s generation with the rigorous temporal semantics of SystemVerilog, effectively “debugging” the model’s logic before the assertion is even finalized.

VI. CONCLUSION AND FUTURE WORK

In this work, we address the semantic gap in LLM-based formal verification by introducing FVRULEARNER, a framework shifting from text imitation to structured, operator-level reasoning. Our experiments identify an operator-level rule saturation phenomenon, where functionality score plateaus once core reasoning rules are learned, regardless of further data scaling. FVRULEARNER outperforms state-of-the-art methods by an average of 31.17% in functional correctness and reduces functional failures by an average of 70.33%, all while maintaining 98.56% syntactic correctness. Additionally, we release NL2SVA-OPENCORE to establish an industrial benchmark for future research. Limited by model reasoning, future work will integrate FVRULEARNER into industrial pipelines and explore post-training (e.g., domain-specific fine-tuning).

ACKNOWLEDGEMENT

This work is supported by NSF 2117997 grant through the A3D3 institute, and Semiconductor Research Corporation (SRC) 2023-CT-3175 grant. We thank Ghaith Bany Hamad for his invaluable and profound technical insights and Syed Suhaib for his support.

REFERENCES

- [1] H. Foster. (2021) Part 8: The 2020 wilson research group functional verification study. Siemens EDA. Accessed: 2024-07-25. [Online]. Available: <https://blogs.sw.siemens.com/verificationhorizons/2021/01/06/part-8-the-2020-wilson-research-group-functional-verification-study/>
- [2] O. Hasan and S. Tahar, "Formal verification methods," in *Encyclopedia of Information Science and Technology, Third Edition*. IGI global, 2015, pp. 7162–7170.
- [3] E. Seligman, T. Schubert, and M. A. K. Kumar, *Formal verification: an essential toolkit for modern VLSI design*. Elsevier, 2023.
- [4] S. Vijayaraghavan and M. Ramanathan, *A practical guide for SystemVerilog assertions*. Springer Science & Business Media, 2005.
- [5] E. Cerny, S. Dudani, J. Havlicek, D. Korchemny *et al.*, *SVA: the power of assertions in systemVerilog*. Springer, 2015.
- [6] C. E. Cummings, "Systemverilog assertions design tricks and sva bind files," in *SNUG*, 2009, pp. 1–42.
- [7] H. Foster, K. Larsen, and M. Turpin, "Introduction to the new accellera open verification library," in *DVCon'06: Proceedings of the Design and Verification Conference and exhibition*. Citeseer, 2006.
- [8] M. Liu *et al.*, "Chipnemo: Domain-adapted llms for chip design," *arXiv:2311.00176*, 2023.
- [9] R. Kande, H. Pearce, B. Tan, B. Dolan-Gavitt, S. Thakur, R. Karri, and J. Rajendran, "Llm-assisted generation of hardware assertions," *arXiv preprint arXiv:2306.14027*, 2023.
- [10] M. Orenes-Vera, M. Martonosi, and D. Wentzlaff, "Using llms to facilitate formal verification of rtl," *arXiv e-prints*, pp. arXiv–2309, 2023.
- [11] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an llm to help with code understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [12] F. Lin, D. J. Kim *et al.*, "When llm-based code generation meets the software development process," *arXiv preprint arXiv:2403.15852*, 2024.
- [13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [14] Z. Wang, Z. Zhou, D. Song, Y. Huang, S. Chen, L. Ma, and T. Zhang, "Where do large language models fail when generating code?" *arXiv preprint arXiv:2406.08731*, 2024.
- [15] W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "Assertllm: Generating and evaluating hardware verification assertions from design specifications via multi-llms," *arXiv preprint arXiv:2402.00386*, 2024.
- [16] OpenCores, "Opencores: Open source hardware designs," OpenCores Website, 2024, accessed: June 28, 2024. [Online]. Available: <https://opencores.org/>
- [17] N. Björner, A. Browne, and Z. Manna, "Automatic generation of invariants and intermediate assertions," *Theoretical Computer Science*, vol. 173, no. 1, pp. 49–87, 1997.
- [18] A. Hekmatpour and A. Salehi, "Block-based schema-driven assertion generation for functional verification," in *14th Asian Test Symposium (ATS'05)*. IEEE, 2005, pp. 34–39.
- [19] D. Yu, H. Foster, and T. Fitzpatrick, "A survey of machine learning applications in functional verification," *DVCon US*, 2023.
- [20] M. J. Kang and H. Ren, "Fveval: Evaluating llms on hardware formal verification," <https://github.com/NVlabs/FVEval>, 2024.
- [21] V. Pulavarthi, D. Nandal, S. Dan, and D. Pal, "Assertionbench: A benchmark to evaluate large-language models for assertion generation," *arXiv preprint arXiv:2406.18627*, 2024.
- [22] C. Sun, C. Hahn, and C. Trippel, "Towards improving verification productivity with circuit-aware translation of natural language to systemverilog assertions," in *First International Workshop on Deep Learning-aided Verification*, 2023.
- [23] B. Mali, K. Maddala, S. Reddy, V. Gupta, C. Karfa, and R. Karri, "Chiraag: Chatgpt informed rapid and automated assertion generation," *arXiv preprint arXiv:2402.00093*, 2024.
- [24] M. Hassan, S. Ahmadi-Pour, K. Qayyum, C. K. Jha, and R. Drechsler, "Llm-guided formal verification coupled with mutation testing," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–2.
- [25] K. Maddala, B. Mali, and C. Karfa, "Laag-rv: Llm assisted assertion generation for rtl design verification," in *2024 IEEE 8th International Test Conference India (ITC India)*. IEEE, 2024, pp. 1–6.
- [26] W. Xiao, D. Ekberg, and R. Karri, "Hybrid-nl2sva: Integrating rag and finetuning for llm-based nl2sva," *arXiv preprint arXiv:2506.21569*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.21569>
- [27] J. Cen, J. Liu, Z. Li, and J. Wang, "Sqlfixagent: Towards semantic-accurate sql generation via multi-agent collaboration," *arXiv preprint arXiv:2406.13408*, 2024.
- [28] M. Endres, S. Fakhoury, S. Chakraborty, and S. K. Lahiri, "Can large language models transform natural language intent into formal method postconditions?" *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1889–1912, 2024.
- [29] F. He, J. Zhai, and M. Pan, "Beyond code generation: Assessing code llm maturity with postconditions," *arXiv preprint arXiv:2407.14118*, 2024.
- [30] N. Mündler, M. N. Müller, J. He, and M. Vechev, "Code agents are state of the art software testers," *arXiv preprint arXiv:2406.12952*, 2024.
- [31] J. Song, J. Raiman, and B. Catanzaro, "Effective large language model debugging with best-first tree search," *arXiv preprint arXiv:2407.19055*, 2024.
- [32] X. Yin, C. Ni, S. Wang, Z. Li, L. Zeng, and X. Yang, "Thinkrepair: Self-directed automated program repair," *arXiv preprint arXiv:2407.20898*, 2024.
- [33] C.-T. Ho and H. Ren, "Large language model (llm) for standard cell layout design optimization," *arXiv preprint arXiv:2406.06549*, 2024.
- [34] Anthropic PBC, "Introducing claude sonnet 4.5," <https://www.anthropic.com/news/claude-sonnet-4-5>, Sep. 2025, accessed: 2025-11-06.
- [35] DeepSeek-AI, "Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence," *arXiv preprint arXiv:2406.11931*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.11931>
- [36] Y. Tsai *et al.*, "Rtlfixer: Automatically fixing rtl syntax errors with large language models," *arXiv:2311.16543*, 2023.
- [37] C.-T. Ho *et al.*, "Verilogcoder: Autonomous verilog coding agents with graph-based planning and ast-based waveform tracing tool," in *AAAI*, 2025. [Online]. Available: <https://doi.org/10.1609/aaai.v39i1.32007>
- [38] J. Wan, C.-T. Ho, R. Liang, C. Yu, D. Chen, and H. Ren, "Schemacoder: Automatic log schema extraction coder with residual q-tree boosting," *arXiv preprint arXiv:2508.18554*, 2025, preprint. URL: <https://arxiv.org/abs/2508.18554>.
- [39] Z. He *et al.*, "Chateda: A large language model powered autonomous agent for eda," in *MLCAD*, 2023.
- [40] X. Liang, S. Song, Z. Zheng, H. Wang, Q. Yu, X. Li, R.-H. Li, F. Xiong, and Z. Li, "Internal consistency and self-feedback in large language models: A survey," *arXiv preprint arXiv:2407.14507*, 2024.
- [41] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, "Automatically correcting large language models: Surveying the landscape of diverse self-correction strategies," *arXiv preprint arXiv:2308.03188*, 2023.
- [42] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, "When can llms actually correct their own mistakes? a critical survey of self-correction of llms," *arXiv preprint arXiv:2406.01297*, 2024.
- [43] N. Jiang, X. Li, S. Wang, Q. Zhou, S. B. Hossain, B. Ray, V. Kumar, X. Ma, and A. Deoras, "Training llms to better self-debug and explain code," *arXiv preprint arXiv:2405.18649*, 2024.
- [44] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [45] H. Huang, Z. Lin, Z. Wang, X. Chen, K. Ding, and J. Zhao, "Towards llm-powered verilog rtl assistant: Self-verification and self-correction," *arXiv preprint arXiv:2406.00115*, 2024.
- [46] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [47] Y.-C. Liao, C.-J. Yu, C.-Y. Lin, H.-F. Yun, Y.-H. Wang, H.-M. Li, and Y.-C. Fan, "Learning-from-mistakes prompting for indigenous language translation," *arXiv preprint arXiv:2407.13343*, 2024.
- [48] Y. Qu, T. Zhang, N. Garg, and A. Kumar, "Recursive introspection: Teaching language model agents how to self-improve," *arXiv preprint arXiv:2407.18219*, 2024.
- [49] Y. Tong, D. Li, S. Wang, Y. Wang, F. Teng, and J. Shang, "Can llms learn from previous mistakes? investigating llms' errors to boost for reasoning," *arXiv preprint arXiv:2403.20046*, 2024.
- [50] T. Zhang, A. Madaan, L. Gao, S. Zheng, S. Mishra, Y. Yang, N. Tandon, and U. Alon, "In-context principle learning from mistakes," *arXiv preprint arXiv:2402.05403*, 2024.

- [51] H. Sun, Y. Jiang, B. Wang, Y. Hou, Y. Zhang, P. Xie, and F. Huang, "Retrieved in-context principles from previous mistakes," *arXiv preprint arXiv:2407.05682*, 2024.
- [52] X. Liang, S. Niu, S. Zhang, S. Song, H. Wang, J. Yang, F. Xiong, B. Tang, C. Xi *et al.*, "Empowering large language models to set up a knowledge retrieval indexer via self-learning," *arXiv preprint arXiv:2405.16933*, 2024.
- [53] X. Yang, H. Chen, W. Feng, H. Wang, Z. Ye, X. Shen, X. Yang, S. Sun, W. Liu, and J. Bian, "Collaborative evolving strategy for automatic data-centric development," *arXiv preprint arXiv:2407.18690*, 2024.
- [54] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," 2024.
- [55] S. Zhang, J. Zhang, J. Liu, L. Song, C. Wang, R. Krishna, and Q. Wu, "Offline training of language model agents with functions as learnable weights," in *Forty-first International Conference on Machine Learning*, 2024.
- [56] W. Zhou, Y. Ou, S. Ding, L. Li, J. Wu, T. Wang, J. Chen, S. Wang, X. Xu, N. Zhang *et al.*, "Symbolic learning enables self-evolving agents," *arXiv preprint arXiv:2406.18532*, 2024.
- [57] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "Textgrad: Automatic" differentiation" via text," *arXiv preprint arXiv:2406.07496*, 2024.
- [58] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in neural information processing systems*, vol. 36, pp. 46 595–46 623, 2023.
- [59] Cadence Design Systems, "Cadence jaspergold formal verification platform," https://www.cadence.com/en_US/home/tools/system-design-and-verification/formal-and-static-verification/jasper-gold-verification-platform.html, 2024, accessed: 2024-07-31.
- [60] J. Niimi, "A simple ensemble strategy for llm inference: Towards more stable text classification," in *International Conference on Applications of Natural Language to Information Systems*. Springer, 2025, pp. 189–199.
- [61] OpenAI *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [62] OpenAI, "Openai o3-mini," <https://openai.com/index/openai-o3-mini/>, Jan. 2025, accessed: 2025-11-06.